

Reinforcement Learning

Algorithms

DQN

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

loss function是基于动作价值的贝尔曼最优方程和TD学习的，梯度下降更新

offline policy，行为策略是 ε -greedy，这样既能探索，又能利用当前最优动作；目标策略是greedy策略，即选择最大Q值动作，可以利用Replay-buffer学习。

DQN的策略是选择最大Q值的动作，只能处理离散动作空间。（但是状态空间是连续的，不像Q-learning）

DQN的目标策略更新是硬更新，n步进行一次，而不是后来那些（如DDPG、SAC等）采用的软更新（ema更新）

PPO

关于PPO与vanilla policy gradient的区别：

policy-gradient:

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$$

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$$

而PPO(暂时未clip版本，也就是CPI目标)与这个方向某种程度上完全一样：

$$\hat{g} = \hat{\mathbb{E}}_t \left[\frac{\nabla_{\theta} \pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t \right]$$
$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t \right]$$

区别只在于PG没有区分 θ_{old} 与 θ ，只要把PPO的 θ_{old} 也变成 θ 就会得到一模一样的公式。这说明它们的更新方向和梯度原理本质一样，但是策略更新都伴随着一定程度的distribution shifting，只有在Trust Region之内才能保证更新不会因为剧烈的分布偏移而失效

同样因为分布偏移的问题，策略梯度算法只能是on-policy的算法，并且因为没有考虑trust region，容易崩溃，因此论文原文说“vanilla policy gradient methods have poor data efficiency and robustness”，以及“While it is appealing to perform multiple steps of optimization on this loss L^{PG} using the same trajectory, doing so is not well-justified, and empirically it often leads to destructively large policy updates”，而PPO则因为已经提前限制了策略更新跨度，所以可以在一定的epochs内反复利用一轮采样数据，成为了半在线半离线(严格说还是on-policy)的策略，并且相对更鲁棒

PPO:

$$L^{CPI}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t \right] = \hat{\mathbb{E}}_t \left[r_t(\theta) \hat{A}_t \right]$$
$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

PPO - Clip Surrogate objective

$$L^{CLIP}(\theta) = \hat{E}_t[\min(\underbrace{r_t(\theta)\hat{A}_t}_{\text{CPI}}, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

四个情况

$\hat{A}_t > 0, \text{ratio} > 1+\epsilon$:

$\hat{A}_t < 0, \text{ratio} < 1-\epsilon$:

$\hat{A}_t > 0, \text{ratio} < 1-\epsilon$:

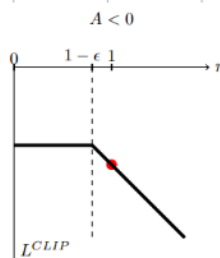
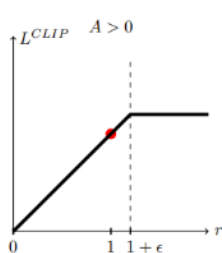
$\hat{A}_t < 0, \text{ratio} > 1+\epsilon$:

被 clip 到 $1+\epsilon$, 防止过分更新 - PPO 中 CLIP 的梯度不考虑 (进 CLIP 就不更新)

折衷略选 好动作 的概率大幅降低, 需大力补板

折衷略选 坏动作 的概率大幅升高

后两种情况中 $\min()$ 起作用, 采用 CPI 那一项, 可以及时补板



PPO CLIP 的梯度传递情况: 防止过度优化, 但 $A > 0$ 且 $r \ll 1$ 或 $A < 0$ 且 $r \gg 1$ 时不限制

PS:: 严格来说:

PPO 是一种 Policy Optimization / Policy Gradient 方法

而不是一种 Actor-Critic 方法。

其原文的重点在策略网络如何更新而并未过多提及价值网络, 只是在 sec.5 后面附带提及了一些计算 Advantage 的方法, 包括 GAE、finite-horizon estimators 与 learned state-value function $V(s)$ 的结合等等

Algorithm 1 PPO, Actor-Critic Style

```

for iteration=1, 2, ... do
  for actor=1, 2, ..., N do
    Run policy  $\pi_{\theta_{old}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{old} \leftarrow \theta$ 
end for
    
```

SAC

--off-policy、robustness、data efficiency

Although the soft Q-learning algorithm proposed by Haarnoja et al. (2017) has a value function and actor network, it is not a true actor-critic algorithm: the Q-function is

estimating the optimal Q-function, and the actor does not directly affect the Q-function except through the data distribution.

虽然有的算法同时有Actor和Critic网络，但是它们的梯度并不耦合在一起，而是分开优化：

```
td_target = rewards + gamma * critic(next_states)

td_delta = td_target - critic(states)

actor_loss = -log_probs * td_delta.detach()
```

此时策略网络的目的不是定义 Critic，而是：

从当前估计的最优Q函数中采样动作。

the actor network as an approximate sampler, rather than the actor

SPI: soft policy iteration

优化对象：

$$V(s_t) = \mathbb{E}_{a_t \sim \pi} [Q(s_t, a_t) - \log \pi(a_t | s_t)]$$

V的损失函数：

$$J_V(\psi) = \mathbb{E}_{s_t \sim \mathcal{D}} \left[\frac{1}{2} \left(V_\psi(s_t) - \mathbb{E}_{a_t \sim \pi_\phi} [Q_\theta(s_t, a_t) - \log \pi_\phi(a_t | s_t)] \right)^2 \right]$$

贝尔曼迭代过程：

$$\mathcal{T}^\pi Q(s_t, a_t) \triangleq r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})]$$

Q的损失函数：

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - \hat{Q}(s_t, a_t) \right)^2 \right]$$

$\hat{Q}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})]$

其中 $\hat{Q}$ 表示ema更新的V网络

SAC的引理证明了在这种迭代之下Q能够收敛到 π 的价值上

SPI的策略是：

$$\pi_{\text{new}} = \arg \max_{\pi} \mathbb{E}_{a \sim \pi} [Q^\pi(s, a) - \alpha \log \pi(a | s)]$$

而可证明要使 π_{new} 是使得V最大的策略，只需

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot | s_t) \parallel \frac{\exp(Q^{\pi_{\text{old}}}(s_t, \cdot))}{Z^{\pi_{\text{old}}}(s_t)} \right)$$

the policy parameters can be learned by directly minimizing the expected KL-divergence，其中exp表示指数，Z是所有exp(Q)的和

$$Z^{\pi_{\text{old}}}(s_t) = \sum_a \exp(Q^{\pi_{\text{old}}}(s_t, a))$$

动作的优化目标：

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}} \left[D_{\text{KL}} \left(\pi_{\phi}(\cdot | s_t) \parallel \frac{\exp(Q_{\theta}(s_t, \cdot))}{Z_{\theta}(s_t)} \right) \right]$$

而这里的动作 π 是从正态分布中采样

$$a_t = f_{\phi}(\epsilon_t; s_t)$$

并且因为正态分布后梯度不能传递，所以这里采用了重参数化处理， $a_t = \mu_{\theta}(s_t) + \sigma_{\theta}(s_t) * \epsilon_t$ ，使得梯度可以通过正态采样传递

展开得到（注意下式中Z因为与网络参数无关所以可以直接省略掉）

$$J_{\pi}(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}} \left[\log \pi_{\phi}(f_{\phi}(\epsilon_t; s_t) | s_t) - Q_{\theta}(s_t, f_{\phi}(\epsilon_t; s_t)) \right]$$

对 ϕ 求梯度：

$$\hat{\nabla}_{\phi} J_{\pi}(\phi) = \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) + \left(\nabla_{a_t} \log \pi_{\phi}(a_t | s_t) - \nabla_{a_t} Q_{\theta}(s_t, a_t) \right) \nabla_{\phi} f_{\phi}(\epsilon_t; s_t)$$

注意这里策略与价值的网络之间求梯度时没有sg环节，都是耦合的

注意，我们说SAC是一个off-policy算法，是因为SAC的policy是在拟合一个由Q定义的 Boltzmann 分布，它的本质是接近Q-based的，因为Q才是最终决定策略方向的。SAC训练过程中Actor与Critic交替更新，新的policy 不断往新的 Boltzmann 分布靠近（SPI）。

SAC的最终目标::（一方面V网络拟合的就是Q+H，另一方面策略网络的目的就是最大化这个值）

$$\max [Q(s, a) + \alpha \mathcal{H}(\pi)]$$

QMIX

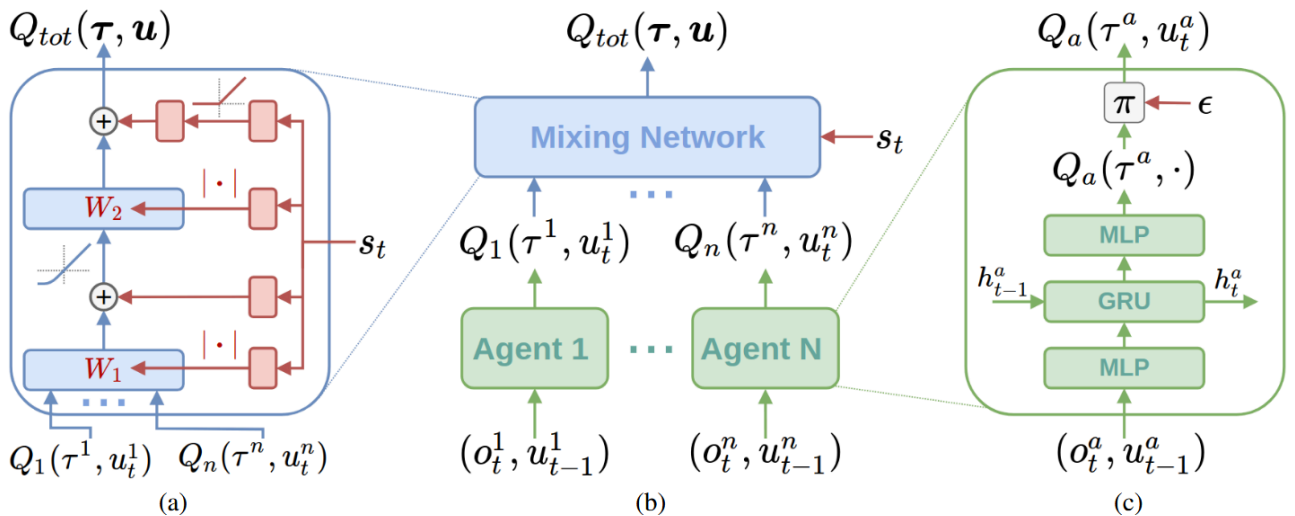


Figure 2. (a) Mixing network structure. In red are the **hypernetworks that produce the weights and biases** for mixing network layers shown in blue. (b) The overall QMIX architecture. (c) Agent network structure. Best viewed in colour.

Centralized Critic & De-centralized Actor

与之对比的另外几种思想：

IQL(Independent Q-Learning):各自为政，各自优化Q function，但是不保证收敛，因为agents同事们的剧烈变化意味着对每个agent而言环境也在剧烈变化

COMA:共享一整个Critic，各自有actor，且这个Critic能看到所有的agent动作和全局state并以此为输入（注意：Q-function的输入是全局状态S和n个agents的联合动作空间，维度很高），在agents数量多维度高的时候训练困难

VDN(Value Decomposition Networks)

同样共享一个 Q_{tot} ，但是这个 Q_{tot} 是由n个 Q_i 简单求和得到的，并且每个 Q_i 的输入只有观测 o_t 和自己的动作空间 u_t ，维度远小于联合动作空间。但是因为简单加和的关系，无法表达agents之间复杂的相互关系。

$$Q_{tot}(\tau, \mathbf{u}) = \sum_{i=1}^n Q_i(\tau^i, u^i; \theta^i)$$

QMIX: value function that can be factored into a non-linear monotonic combination of the agents' individual value functions in the fully observable setting，接近VDN，但是每个 Q_i 是用一个独立的w&b网络组合起来，保证weights非负，而不只是简单地加和

$$\mathcal{L}(\theta) = \sum_{i=1}^b \left[y_i^{\text{tot}} - Q_{tot}(\tau, \mathbf{u}, s; \theta) \right]^2$$

$$y_i^{\text{tot}} = r + \gamma \max_{\mathbf{u}'} Q_{tot}(\tau', \mathbf{u}', s'; \theta^-)$$

其中 θ^- 是EMA更新参数， y_i^{tot} 表示target network。

QMIX没有强调是在离散的动作空间还是连续的，但同理的，离散的策略就是 $Max(Q_{tot})$ ，连续的就像SAC一样用重参数采样，拿个神经网络降kl散度

MAPPO

系统研究了哪些 trick 对多智能体 PPO 真正有效, 本质上是类似COMA的结构，用ppo当actor。（IPPO则是各自用自己的critic）

Value Normalization

优势计算：使用未归一化的V-real，用GAE计算得到V-target

Phase 1 compute_gae — 计算 δ_t 、GAE、 V^{target}

Input: rewards $r_{1:T}$, dones $d_{1:T}$, $V_\phi(s_{1:T+1})$ after denorm, γ , λ

Output: advantages $\hat{A}_{1:T}$, returns $\hat{V}^{\text{target}}_{1:T}$

```
1 with no_grad():
2     denormalize Critic output:  $V_{\text{real}}(s_t) = V_\phi(s_t) \cdot \sigma + \mu$  ▷ 还原真实尺度
3     gae ← 0
4     for t = T, T-1, ..., 1:
5         mask ← 1 - d_t ▷ episode 结束则截断 bootstrap
6          $\delta_t \leftarrow r_t + \gamma \cdot V_{\text{real}}(s_{t+1}) \cdot \text{mask} - V_{\text{real}}(s_t)$  TD error, 真实尺度
7          $\text{gae} \leftarrow \delta_t + \gamma \cdot \lambda \cdot \text{mask} \cdot \text{gae}$  ▷ 递推累积优势
8          $\hat{A}_t \leftarrow \text{gae}$ 
9      $\hat{V}^{\text{target}}_t \leftarrow \hat{A}_t + V_{\text{real}}(s_t)$  真实尺度的回归目标
10 return  $\hat{A}_{1:T}$ ,  $\hat{V}^{\text{target}}_{1:T}$ 
```

得到V-target后进行归一化，再由网络预测，也就是critic的预测内容是归一化后的价值

Phase 2 critic_loss_with_normalization — 归一化 + 回归

Input: $s_{1:T}$, $\hat{V}^{\text{target}}_{1:T}$ (from Phase 1), running stats (μ , σ)

Output: L_{critic} , ∇_ϕ

```
1 with no_grad(): ▷ 更新统计量，不进入计算图
2      $\mu, \sigma \leftarrow \text{RunningUpdate}(\mu, \sigma, \hat{V}^{\text{target}})$  Welford 在线更新
3      $\hat{V}^{\text{norm}} \leftarrow (\hat{V}^{\text{target}} - \mu) / \sigma$  ▷ stop_gradient, 均值  $\approx 0$  方差  $\approx 1$ 
4      $V^{\text{norm}}_{\text{pred}} \leftarrow \text{Critic}_\phi(s)$  ▷ 接入计算图
5      $L_{\text{critic}} \leftarrow 1/T \cdot \sum_t (V^{\text{norm}}_{\text{pred},t} - \hat{V}^{\text{norm}}_t)^2$ 
6      $\nabla_\phi \leftarrow 2/T \cdot \sum_t (V^{\text{norm}}_{\text{pred},t} - \hat{V}^{\text{norm}}_t) \cdot \partial V_\phi(s_t) / \partial \phi$ 
7 return  $L_{\text{critic}}$ 
```

个人感觉这种线性的归一化，作用一定都是为了解决不同来源/不同阶段的信号幅度不一致问题，如flow matching目标的归一化是为了同一多种数据集中的动作标签的幅度差异，让它能从不同的数据集中学到一样的共性。那么对于这种方差均值的归一化，假设全程收集到的V-real方差均值都不变，则这个归一化理论上不会有什么作用（只是把数值整体平移放缩而已），但是即使 r 的分布完全稳定， V_{target} 也会漂移，因为策略变好了，同样的状态能拿到更多累积回报， $V(s)$ 估计值整体抬升。所以归一化的动机根本上是训练动态导致的非平稳性，而不是数据分布本身的方差问题。并且这种归一化不会改变V的相对大小，所以不会影响学习的大方向。但实际上这种处理确实压缩了一些信息（比如V变好的

程度)，只是基于对训练平稳性的权衡而采用的trick，不能让它学到更多有用的内容
遇到 reward 或 value 尺度不稳定的情况，判断要不要加归一化的标准就是这个：被压缩掉的信息对任务是否真的无关紧要。

Input Representation to Value Function

Value-function的几种输入内容的效果差异

IND：只输入每个 agent 的 Critic 只用自己的局部观测 o_i

EP (Environment-Provided)：用环境提供的全局状态向量

AS (Agent-Specific)：把 IND 和 EP **直接拼接**，每个 agent 的 Critic 同时看到：自己的局部观测 + 环境全局状态

FP (Feature-Pruned)：从 AS 出发，去掉重叠的特征，只保留互补信息，是论文推荐的方案

epoch数、minibatch数、batch大小

总结：epoch和minibatch数都不应太大，因为MARL中本身环境变化比较剧烈，epoch大了会导致策略变化过大，minibatch过多会因为每个batch太小而导致方差较大，不利于优化而batch较大总体上对训练有益，但太大会降低data efficiency

关于RL的几个主要分类

on policy Vs off policy

on：策略梯度的那些、PPO

off：DQN、DDPG、SAC

判断是否off-policy的标准是 看Actor是怎么更新的

普通的策略梯度：

$$\nabla_{\theta} J = \mathbb{E}_{a \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a)]$$

要求 a 必须来自 π_{θ} 的采样，否则梯度就不对了：

$$\mathbb{E}_{a \sim \pi_{\text{old}}} [\nabla \log \pi_{\text{new}}(a|s) A]$$

根本原因是PG直接基于当前各个动作的概率分布做梯度，因而也要求采样的动作概率必须与当前的策略的概率分布一致

与之形成对比的是DDPG这种：

$$\nabla_{\theta} J = \mathbb{E}_{s \sim \rho^{\beta}} [\nabla_a Q^{\mu}(s, a) \Big|_{a=\mu_{\theta}(s)} \cdot \nabla_{\theta} \mu_{\theta}(s)]$$

训练的是基于状态和价值函数应该输出什么样的动作，而不需要考虑这个标签动作是否来自当前的策略（呃实际上DDPG的replay buffer中的动作样本只用来训练critic了跟actor没啥关系）

那么同理的，SAC亦然，actor更新时只有state来自replay buffer，根本不依赖 a_{buffer}

最深层的理解

其实可以把三者统一起来：

PPO

Actor学：

旧动作好不好

所以必须看 Buffer 动作。

DDPG

Actor学：

Q最大的动作是什么

自己生成动作即可。

SAC

Actor学：

Q诱导的Boltzmann分布是什么

自己采样动作即可。

model-based Vs model-free

连续动作空间 Vs 离散动作空间

连续：

SAC、DDPG(直接输出动作本身，也不需要像SAC那样重参数化，也不是像离散网络那样输出某个动作的概率，但也因此是确定性策略)

离散：

Q-learning(状态空间也离散)、sarsa(TD)、DQN、REINFORCE(策略梯度)
(前面两种都属于是建表来记录Q值或记录其迭代过程的那种)

value-based Vs policy-based

基于价值：

如DQN、Q-learning等等，求得一个价值函数，策略就是找价值最大的动作

基于策略：

$$\begin{aligned} J(\theta) &= \mathbb{E}_{s_0}[V^{\pi_\theta}(s_0)] \\ \nabla_\theta J(\theta) &\propto \sum_{s \in S} \nu^{\pi_\theta}(s) \sum_{a \in A} Q^{\pi_\theta}(s, a) \nabla_\theta \pi_\theta(a|s) \\ &= \sum_{s \in S} \nu^{\pi_\theta}(s) \sum_{a \in A} \pi_\theta(a|s) Q^{\pi_\theta}(s, a) \frac{\nabla_\theta \pi_\theta(a|s)}{\pi_\theta(a|s)} \\ &= \mathbb{E}_{\pi_\theta}[Q^{\pi_\theta}(s, a) \nabla_\theta \log \pi_\theta(a|s)]. \end{aligned}$$

这是策略梯度的通用公式，区别在于用什么去估计此处的Q值（其实不止是Q，只要期望等于total return，这是RL优化的根本目标），如Monte Carlo、GAE、TD形式，优势函数形式等等，都是为了平衡偏差与方差的方法。注意这里的 π_θ 既是行为策略也是目标策略，既是采样所用的策略也是最终要优化的策略（on-policy）因为这个梯度的期望是基于这个策略求得，如果策略不一样会导致价值函数不匹配

Actor-Critic

既学习价值函数，又学习策略函数

价值函数更新梯度：

$$\begin{aligned} \mathcal{L}(\omega) &= \frac{1}{2} (r + \gamma V_\omega(s_{t+1}) - V_\omega(s_t))^2 \\ \nabla_\omega \mathcal{L}(\omega) &= - (r + \gamma V_\omega(s_{t+1}) - V_\omega(s_t)) \nabla_\omega V_\omega(s_t) \end{aligned}$$

这是其中的一种形式，基于时序差分的迭代，其中减号前面部分是目标部分，属于s·g部分策略函数更新的部分就是基于这里学到的价值函数，带入上面的策略梯度去更新策略网络